

MemoryGraph AI: Context-Aware Hybrid Knowledge Graphs

Amrutha M
B.Tech(H), School of CSE
RV University, Bengaluru, India
amrutham.btech22@rvu.edu.in

Bhargava S
B.Tech(H), School of CSE
RV University, Bengaluru, India
bhargavas.btech22@rvu.edu.in

Renu Bojja
B.Tech(H), School of CSE
RV University, Bengaluru, India
renub.btech22@rvu.edu.in

Dr. Shouvik Chakraborty
Assistant Professor, School of CSE
RV University, Bengaluru, India
shouvikchakraborty@rvu.edu.in

Abstract—Conventional keyword based and vector based retrieval approaches often struggle to capture explicit semantic relationships of the concepts, making them unsuitable for multi-hop reasoning across multiple documents. This paper presents MemoryGraph AI, a hybrid knowledge graph framework that is able to perform multi-document question answering and knowledge discovery. It extracts entities and their relationships from unstructured documents using large language models and constructs a structured knowledge graph that resides in Neo4j. To increase the ability of the system to retrieve information, MemoryGraph uses a combined retrieval method that utilizes a graph traversal method and a vector similarity search through the use of SentenceTransformer embeddings, allowing contextually aware retrieval and cross-document reasoning.

Additionally, as compared to traditional retrieval methodologies, the proposed framework maintains a persistent and incrementally evolving knowledge graph, allowing knowledge extracted from previously processed documents to be used across future queries and sessions. The system also provides graph evidence visualization, showing the relevant entities and relationships used to generate answers, thereby improving transparency and explainability. Additionally, an interactive interface supports document ingestion, knowledge exploration, and query-based graph visualization.

To evaluate system performance, both retrieval quality metrics and pipeline performance metrics are analyzed. Retrieval effectiveness is assessed using MRR, Hit Rate@K, contextual relevance, and answer faithfulness, while system efficiency is evaluated using time-based metrics such as ingestion time, entity extraction time, embedding generation time, total pipeline runtime, and query latency. Experimental results demonstrate consistent improvements of the hybrid retrieval approach over vector-only baselines in both contextual retrieval accuracy and query response efficiency.

These findings highlight the effectiveness of hybrid knowledge graph-based retrieval systems for scalable, domain-independent document analysis and intelligent knowledge discovery in research and enterprise environments.

Keywords—Knowledge Graph, Hybrid Retrieval, Retrieval-Augmented Generation, Neo4j, Large Language Models, Multi-Document Question Answering, Graph Traversal, Vector Embeddings, Information Extraction, Natural Language Processing

I. INTRODUCTION

The massive growth of digital documents, scholarly journals, and technical documentation has resulted in considerable obstacles to performing adequate knowledge discovery. In numerous scientific fields—biomedical sciences, cyber security, and legal analysis—researchers and other professionals often must deal with massive collections of documents that contain

some connection or relationship to one another. Unfortunately, current information retrieval solutions provide insufficient assistance when multiple documents need to be the basis for structured reasoning. Traditional keyword-based search engines and modern vector-based similarity search systems allow for the identification of documents that are likely relevant but do not provide a method for directly representing the semantic representation of different entities over the course of their respective histories. Therefore, retrieving and answering complex queries often requires multiple distinct reasoning processes over a series of documents because conventional similarity-based retrieval systems do not assist substantially in performing these types of activities.

To overcome some of these limitations, Retrieval-Augmented Generation (RAG) has provided an effective means of establishing a foundation for large language model (LLM) outputs from disparate external knowledge sources and reducing the number of hallucinations that generally occur in information-intensive applications [1], [5]. Most standard RAG implementations retrieve document chunks from the text using dense vector embedding techniques and provide them to LLMs as contexts for generating outputs [1], [3]. While effective for the semantic retrieval of documents, these systems typically treat documents as entirely independent items in relation to one another and do not include any form of mechanisms for modeling the relationships between the various entities [13]; therefore, these systems generally cannot help users retrieve and create outputs that use information from multiple documents or provide insight into how concepts are related in general [7], [18].

Knowledge Graphs are formal representations for describing information. They treat entities as nodes, with edges connecting the nodes based on relationships. This structure provides a natural form for understanding how to perform multi-hop reasoning and contextually infer connections among related items [17]. Many recent studies have looked at hybrid methods that integrate knowledge graph traversals with vector retrieval methods [6], [17]. Many current systems create knowledge graphs as part of static preprocessing and then discard them after each retrieval session, making it impossible to accumulate knowledge over time [18]. Most current methods also operate on a single document level by generating individual graphs for each document, rather than generating one unified complete knowledge graph containing all entities and relationships from

all documents [4], [10].

This paper introduces a MemoryGraph AI which uses a combination of knowledge graphs and semantic search using documents. This technique uses large language models to extract unique relationships and entities from the multiple documents processed during each interaction to create a persistent, continuously evolving memory graph or knowledge graph comprising of knowledge pulled from all documents processed. This memory graph serves as a one-stop shop for entities and their interrelationships that can be accumulated over time through continued processing of the same documents. There is also an integration of entities across multiple documents using a method of combining semantically related extracted entities found in all the documents to create a comprehensive knowledge representation. MemoryGraph AI employs two distinct retrieval methods: (1) a vector space similarity search of entity representations, and (2) traversing the knowledge graph to answer questions using both structural and semantic relationships between the entities. Experimental results demonstrate that the implementation of hybrid retrieval architecture for MemoryGraph AI produced significantly better quality of retrieval and reasoning capabilities compared to the vector-only baseline across several retrieval performance metrics.

II. LITERATURE SURVEY

A. Review of Existing Research Work

Recent progress in the field of information retrieval and question answering from document collections has focused on leveraging knowledge graphs, language models, and retrieval-augmented generation [1] [13] [18]. Existing approaches use vector space methods to retrieve the information needed for solving a problem [3]. Even though they perform well on retrieving semantically similar documents, they do not account for the structure of knowledge, making it hard for them to reason in multi-hop scenarios. Thus, the usage of knowledge graph embeddings is essential in recent approaches to solve this challenge.

One such approach was developed by Lewis et al., referred to as retrieval-augmented generation [1]. It uses embeddings to identify relevant documents through dense retrieval and generate answers conditionally based on this context. Despite its success in generating knowledge-rich information, RAG does not account for any structural aspects of the information retrieved from multiple documents. Hence, it fails when used to solve multi-hop problems.

Graph augmentation-based retrieval architectures like KG2RAG and GeAR have been introduced in response to these issues [17] [18]. KG2RAG exploits the power of knowledge graph traversals to enrich and arrange the retrieved context while GeAR utilizes a novel agent-based framework for iterative multi-hop retrieval based on semantics [17]. The advantages of graph-based models for retrieval tasks are evident in these works [17]. However, these models operate on static graphs generated during the preprocessing phase,

thereby reducing their flexibility and scalability in relation to new documents and changing queries.

Similar advances in constructing knowledge graphs include Docs2KG, AutoKG, and RAKG [18] where LLMs, OpenIE, and contextual entity linking is used for automated knowledge graph construction. Though the approaches enhance knowledge graph accuracy, their applications have mostly been restricted to individual documents and not to the development of a continuous, cross-document knowledge graph. Systems like HybridRAG and MegaRAG combine graph reasoning with vector retrieval, achieving better results [18]. Nevertheless, surveys by Choi et al. and Zhong et al. bring out certain issues faced by researchers in these aspects [13] [18].

B. Comparative Analysis

The current state-of-the-art solutions for document-based QA can be broadly divided into three groups: keyword-based methods, vector similarity searches, and graph-based reasoning. The keyword-based approach is computationally efficient, yet does not provide any semantic awareness. The vector embedding method provides efficient semantic similarity calculations, yet misses relations between objects explicitly stated in the knowledge base [3] [13]. The graph-based reasoning is an approach that works well for relational reasoning tasks [17], yet fails to find related nodes if the query language differs from the graph's terminology.

Comparative reviews reveal trade-offs between scalability, semantic understanding, and reasoning ability as fundamental to each system. Vector databases excel at semantic search, while knowledge graphs provide the structured reasoning needed for complex relationship-driven queries. All three systems lack persistent knowledge memory, weak cross-document integration, and poorly developed graph-vector fusions as persistent deficiencies across systems.

C. Research Gap

Research indicates that the area of document understanding and question answering has made great strides in the integration of knowledge graphs, retrieval systems, and large language models. Previous research shows that graph-based retrieval can provide support for improving multi-hop reasoning, and automated construction of knowledge graphs allows for better organized representations of entities obtained from unorganized documents. Despite these improvements, there remain many critical shortcomings.

One of the major shortcomings to date has been a lack of a persistent memory for knowledge. Current retrieval systems build knowledge graphs during a pre-retrieval process and use them only for a short period of time during that retrieval and then remove or destroy them after the retrieval process. Because knowledge that has been obtained from previously processed documents cannot be used again for future queries, knowledge graphs do not allow systems to build a continual body of knowledge.

Another significant limitation is how little current systems integrate information across document boundaries. In many

frameworks, knowledge graphs are built for each individual document resulting in disconnected structures that do not provide a means for recognizing relationships between entities that originated from different documents. This lack of connection and direction between entities based on what was received from outside the source, decreases a system’s ability to use reliable multi-hop reasoning on data that is located in multiple documents.

A number of current retrieval systems provide limited explainability (of how a response was produced) and limited inference opportunities (in terms of understanding how the entities and relationships in the knowledge graph have been used to arrive at the final answer). Even though retrieval systems use the context of the retrieved documents to create their answers, they do not provide sufficient explicit evidence to show the contribution of the entity and relationship data in the underlying knowledge graph to the final response. Without visualizing this query-level graph evidence, it is very difficult for users to trace the reasoning path that the system has followed.

Furthermore, the majority of proposed methods have been evaluated only against domain specific datasets [2] [14] [20], limiting their use to heterogeneous multi-domain document collections. This further constrains the generalizability of these solutions to real-world applications, because real-world documents exist in a variety of domains and formats.

These limitations underscore the need for a retrieval system that has the ability to retain a persistent and reliable memory of knowledge, integrate the entities and relationships of knowledge across the multiple documents used for retrieval, and provide an explainable graph-based reasoning process that enables users to gain enhanced transparency regarding how the entity and relationship data from the underlying knowledge graph were used during the retrieval process. The proposed system addresses all three of these limitations by creating a persistent, dynamic graph of knowledge from heterogeneous documents that can be used to integrate entities across documents and provide visualization of the query-level graph evidence, thereby improving the contextual reasoning and knowledge discovery capabilities for users of the retrieval system.

III. PROPOSED METHODOLOGY

A. Document Ingestion and Preprocessing

The document ingestion module is able to process documents in multiple formats including PDF, DOCX, and TXT. Documents ingested by the system undergo cleaning to get rid of any non-printable characters, whitespace, and formatting elements. Due to computational constraints imposed by large documents, document text is split into overlapping chunks to facilitate efficient processing while preserving contextual information during extraction.

B. Entity and Relationship Extraction

Once preprocessed, structured information is extracted from documents in the form of entities and relations. A large

language model extracts entities in the document text that include concepts, research methodologies, datasets, tasks, and evaluation criteria among others together with the relations between these entities. Information about entities and relationships are structured for constructing the knowledge graph.

C. Knowledge Graph Construction and Persistent Storage

The information about entities and relationships is then stored in a graph database in the form of a Neo4j graph database. Nodes represent entities while edges capture their relationships. The graph database uses deduplication procedures to combine duplicate entities and thus facilitates the building of a knowledge graph from data across different documents. A persistent graph database allows continuous growth in size by adding additional documents, thus facilitating continuous knowledge accumulation.

$$G = (V, E) \quad (1)$$

D. Graph Embeddings and Vector Indexing

In order to allow semantic search operations, vectors are created for nodes in the graph based on a SentenceTransformer architecture. These vectors help represent semantic similarity and can be stored in the graph database in conjunction with a vector index.

$$\mathbf{v}_i = f_{\theta}(v_i) \in R^d, \quad d = 384 \quad (2)$$

E. Hybrid Retrieval and Query Processing

Upon receiving a query q from the user, a query vector v_q is created by the system to perform semantic similarity searches among entity vectors. Similarity computations between two vectors are done via cosine similarity:

$$\text{sim}(v_q, v_i) = \frac{v_q \cdot v_i}{||v_q|| ||v_i||} \quad (3)$$

High similarity values for entities can help to traverse the graph to get additional information from connected nodes and relationships to understand the context better.

F. Graph Evidence and Visualization

In order to make our approach more understandable and transparent, graph evidence is provided for all user queries. The subgraph representing all entities and relationships obtained during the process of retrieval for each request is visualized, showing the user the way the system came up with its reasoning and providing an understanding of how various concepts are connected through the knowledge relationship network.

G. Conversational Interface and Follow-Ups

The system uses the conversational approach that takes into account the history of previous queries and answers in order not to lose track of context across conversations with the system. Besides that, follow-up questions are suggested after every generated response.

H. Knowledge Discovery Using Graph Analytics

Apart from answering queries, the system enables knowledge discovery by performing graph analytics on the created graph structure. It performs analysis such as calculating hub nodes for high connectivity nodes, and recognizing concept communities within the graph. This analytics capability allows gaining deeper understanding of relationships among the concepts.

I. User Interface Design

This project has an interactive user interface that serves various purposes such as enabling document loading, executing queries, graph visualization and analytics. The interface incorporates several modules such as a module for document upload, query conversation module, graph visualization, and dashboard for system performance.

J. Metrics Monitoring of Pipeline

Metrics such as the number of files processed, ingestion time, extraction time, time for generating embeddings, and pipeline execution time are recorded by the system per each run. Query response time is measured as well while interacting with the user. Such metrics are presented to the user on the metrics dashboard, providing an opportunity to assess the efficiency of the system as well as compare different approaches to data retrieval.

K. System Summary

The suggested approach to building MemoryGraph-based AI framework involves ingestion of multiple documents, knowledge extraction, creation of persistent knowledge graph from the extracted entities, hybrid retrieval based on graph traversal and semantic vector search, and visualization to enable interactive questioning. Additional features of the system include visualization of graph evidence, conversational mode, analysis of knowledge discovery results, and performance monitoring.

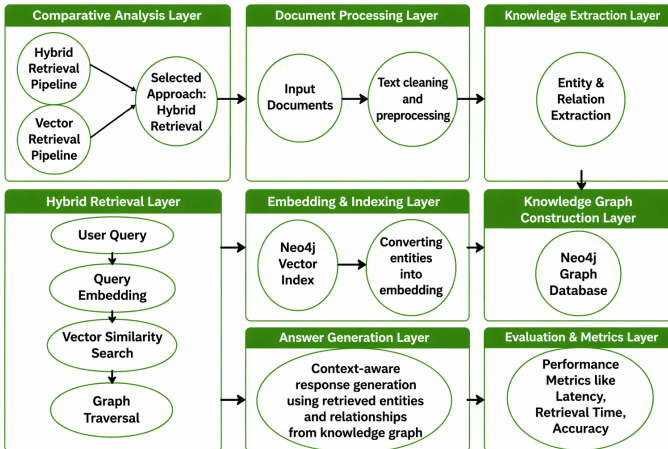


Fig. 1. System architecture of the proposed MemoryGraph AI

IV. EXPERIMENTS AND RESULTS

The experiment seeks to investigate the efficacy of the proposed hybrid Graph RAG architecture against the traditional vector RAG approach. The objective is to identify the role that knowledge graph navigation plays in enhancing the process of document retrieval, and subsequently, answer generation from the retrieved documents. This comparative evaluation is done by putting both architectures under the same condition of using the same knowledge base, same embedding representation, and the same set of queries for a valid assessment.

The evaluation will be carried out based on two levels of criteria. On the first level, system performance measures will be considered. These include measures of ingestion time, entity extraction time, embedding generation time, overall pipeline processing time, and query time. Simultaneously, the retrieval performance will be assessed using measures of MRR, Hit Rate at K, and Entity Coverage.

In order to investigate the influence of retrieval quality on a practical level, an LLM-as-Judge setting is used to evaluate both the generated responses and the context retrieved. The quality of the answer will be determined by measuring the relevance, fidelity, and completeness of the answer, while the quality of the context will be assessed in terms of sufficiency and richness in structure.

A. Experiments Conducted

TABLE I
COMPARISON OF HYBRID RAG VS VECTOR RAG

Metric	Hybrid RAG	Vector RAG
MRR	0.787	0.787
HitRate@K	1	1
EntityCoverage	0.954	0.467
Relevance [1–5]	4.22	4.22
Faithfulness [1–5]	3.89	3
Completeness [1–5]	3.78	3
Sufficiency [1–5]	4.33	3.89
Structural Richness [1–5]	4.11	2.67

The experiment was carried out through an evaluation procedure which was performed under a controlled setting. The manually generated knowledge graph contained 47 nodes and 64 relations as its database. The two models employed similar TF-IDF-weighted bag-of-words embeddings with entity-name boosting, such that the difference between them can be attributed only to their graph traversal mechanisms.

In both methods, the initial step of the retrieval process is performed using the Approximate Nearest Neighbors (ANN) search algorithm to find the top-5 seed entities that share the

highest cosine similarity with the query embedding. However, while Vector RAG retrieves the seed entities without further processing, Graph RAG proceeds with retrieving the one-hop neighbors of each of these seed entities in the knowledge graph by following the outgoing and incoming edges. The 12 most relevant relational triples are chosen and structured into context as follows:

$$(\text{Entity A}) \xrightarrow{\text{RELATION}} (\text{Entity B})$$

The structured representation is then fed into the language model that generates the answers.

In total, eight test queries were created to cover the three reasoning types: relationship-reasoning queries, where one has to be aware of the relationships between entities to generate an answer; traversal-requiring queries, where the directly relevant entity lies a few graph hops away from the ANN seeds; and multi-hop queries, which require the traversal of relationship chains on several entities to retrieve the correct answer. A ground truth answer along with a list of ground-truth relevant entities were annotated for each test query. The answer generation model employed was llama-3.3-70b-versatile, while the LLM judge was llama-3.1-8b-instant.

TABLE II
PERFORMANCE METRICS FOR A SAMPLE QUERY

Metric	Value	Analysis
Files	4	Number of documents uploaded and processed in the pipeline.
Ingest (s)	12.8	Time taken to load and prepare the documents for processing.
Extract (s)	448.41	Time required to extract entities and relationships from the documents to build the knowledge graph.
Embed (s)	58.76	Time taken to generate vector embeddings for document chunks.
Total (s)	520.04	Overall time taken for the entire pipeline execution (ingestion + extraction + embedding).
Latency (s)	0.78	Time taken by the system to generate an answer for a user query after processing.

Table II shows how pipeline times are divided amongst the various steps within each processing step during a typical operation where four files have been uploaded. It is evident from the data that the extraction process occupies the majority of the time, with the processes of embedding creation and document upload being relatively less resource intensive. Latency of queries, which are processed independently of the pipeline process, is less than a second.

B. Analysis and Discussion

The test results prove the superiority of the hybrid Graph RAG model to the baseline Vector RAG model based on the examined measures. Even though the models have the same MRR value, equal to 0.787, and the same value for the Hit Rate@K measure, being equal to 1.0, which implies their similar effectiveness in terms of finding the most relevant document, the discrepancies in other quality metrics are evident.

The most pronounced enhancement can be seen in terms of Entity Coverage, with the Graph RAG model scoring 0.954, whereas the Vector RAG model scores 0.467, with an improvement rate of 104%. In other words, the above result provides a quantitative proof of the value of graph traversal by enabling Graph RAG to retrieve structurally related yet semantically different entities from the query-entities through graph traversal and type-aware edge navigation. On the contrary, due to its restriction to top-5 cosine similarity results, the Vector RAG retrieves less than half of the ground-truth entities among the nine queries.

According to the LLM-as-Judge measure for assessing context quality, Graph RAG receives a Structural Richness score of 4.11 compared to Vector RAG’s 2.67, showing the maximum point difference of 1.44 out of 5 among all other measures used for evaluation. While this metric evaluates how well the context indicates the connectivity of ideas, the latter measure is concerned with just the presence of concepts. Since the extracted context is represented in a relational triple structure resulting from graph traversal, it becomes clear what types of semantic relationships are present in the graph, thus allowing the language model to make factual connections, not hallucinations based on plain entity names.

At the answer generation stage, Graph RAG reaches a Faithfulness score of 3.89 versus 3.00 in Vector RAG and a Completeness score of 3.78 against 3.00 for Vector RAG. This increase in faithfulness means that the grounding in relational triples decreases the likelihood of hallucinations in Vector RAG, which is when the language model comes up with unsupported facts. With regard to completeness, the results show that the context grounded on relations allows for covering all aspects of each query better than the vector one. Regarding Answer Relevance, both systems score 4.22; hence, they demonstrate an equal understanding of the questions.

With respect to system-level efficiency, the values provided in Table II represent a set of performance measures associated with a particular pipeline execution on four documents. Specifically, the total duration of ingestion and extraction operations amounting to 520.04 seconds are primarily consumed by the step of LLM-based entity and relation extraction (448.41 seconds); this process only takes place once per document set and is thus a one-time overhead for further querying. Finally, the average latency associated with performing an actual query equals 0.78 seconds; it includes costs associated with ANN search, graph traversals, and answer generation.

On the whole, it can be seen that there is a distinct balance being achieved here between computational costs and the

output quality of the process. While Graph RAG does bring in some extra computational overhead by virtue of graph traversal, the fact of the matter remains that the gains that it offers in terms of entity coverage, context structure, answer accuracy, and completeness make it worthwhile.

V. CONCLUSION

This paper presented MemoryGraph AI, a hybrid knowledge retrieval framework that integrates knowledge graph reasoning with vector-based semantic retrieval for multi-document question answering. The system constructs a persistent knowledge graph from uploaded documents by extracting entities and relationships and storing them in a structured graph database. By combining graph traversal with embedding-based similarity search, the framework enables context-aware retrieval and supports cross-document reasoning, allowing the system to synthesize information distributed across multiple sources.

In addition to retrieval capabilities, the framework provides explainability through graph evidence visualization, allowing users to observe the relationships and entities that contributed to the generated response. The system also includes an interactive user interface supporting document ingestion, conversational querying, graph visualization, and pipeline performance monitoring. Graph analytics capabilities further enable users to explore hub entities, concept clusters, and hidden relationships within the document corpus. The experimental evaluation demonstrates the effectiveness of combining graph-based reasoning with semantic embeddings, highlighting the advantages of the hybrid retrieval architecture over purely vector-based approaches.

While the current framework demonstrates promising results for document understanding and knowledge discovery, several enhancements can be explored to extend the capabilities of the proposed system. Large-scale graph optimization techniques such as efficient indexing and caching mechanisms can be incorporated to support very large document collections and dense knowledge graphs while maintaining fast retrieval performance. The entity–relation extraction module can also be extended to perform explicit relationship type detection, allowing the system to classify semantic relationships instead of storing generic connections between entities. Additionally, the framework can be expanded to support multi-modal knowledge integration by extracting structured information not only from textual documents but also from tables, figures, and diagrams. Such extensions would significantly enhance the system’s ability to construct richer knowledge graphs and support more comprehensive knowledge discovery.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to Dr. Shouvik Chakraborty, Assistant Professor at RV University, Bangalore, for valuable guidance, support, and encouragement throughout the research and development of this work.

The authors also acknowledge the School of Computer Science and Engineering at RV University for providing the

necessary infrastructure and resources that enabled the successful completion of this research.

REFERENCES

- [1] P. Lewis, E. Perez, B. Piktus, F. Petroni, A. Karpukhin, V. Goyal, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)**, vol. 33, pp. 9459–9474, Dec. 2020.
- [2] R. Yang, Y. Ning, E. Keppo, M. Liu, C. Hong, D. S. Bitterman, et al., “Retrieval-augmented generation for generative artificial intelligence in medicine,” **Harvard Med. School and Duke-NUS Med. School**, vol. 39, no. 2, pp. 101–113, Mar. 2024.
- [3] C. Xiong, J. Lin, K. Xiong, X. Guo, and S. Zhang, “Precise zero-shot dense retrieval without relevance labels,” in **Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)**, vol. 2, pp. 548–562, Oct. 2023.
- [4] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, “Benchmarking retrieval-augmented generation for medicine,” in **Proc. Assoc. Comput. Linguistics (ACL)**, vol. 1, no. 3, pp. 78–92, Aug. 2024.
- [5] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, “REALM: Retrieval-augmented language model pre-training,” in **Proc. 37th Int. Conf. Machine Learning (ICML)**, vol. 119, pp. 3939–3951, Jul. 2020.
- [6] M. Jeong, J. Sohn, M. Sung, and J. Kang, “Improving medical reasoning through retrieval and self-reflection with retrieval-augmented large language models,” **Bioinformatics**, vol. 40, no. 6, pp. 487–502, Jun. 2024.
- [7] G. Xiong, Q. Jin, X. Wang, M. Zhang, Z. Lu, and A. Zhang, “Improving retrieval-augmented generation in medicine with iterative follow-up questions,” in **Proc. Pacific Symp. Biocomputing**, vol. 30, pp. 172–188, Jan. 2025.
- [8] C. Long, Y. Liu, C. Ouyang, and Y. Yu, “Baicai: A domain-optimized retrieval-augmented generation framework for medical applications,” **arXiv preprint arXiv:2405.12345**, Jul. 2024.
- [9] W. Hiesinger, C. Zakka, A. Chaurasia, R. Shad, A. Dalal, J. Kim, et al., “Almanac: Retrieval-augmented language models for clinical medicine,” **Neurology AI J.**, vol. 52, no. 7, pp. 299–315, May 2023.
- [10] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, “Benchmarking retrieval-augmented generation for medicine,” **arXiv preprint arXiv:2402.13178**, Feb. 2024.
- [11] A. K. Lahiri and Q. V. Hu, “AlzheimerRAG: Multimodal retrieval-augmented generation for PubMed articles,” in **Proc. Toronto Metropolitan Univ. AI Symp.**, vol. 11, no. 3, pp. 55–71, Sep. 2024.
- [12] M. F. Romano, L. C. Shih, I. C. Paschalidis, R. Au, and V. B. Kolachalama, “Large language models in neurology research and future practice,” **Neurology**, vol. 42, no. 6, pp. 213–227, Apr. 2023.
- [13] H. Roberts, J. Wang, and P. Tomczak, “Retrieval-augmented generation for AI-generated content: A survey,” **Artif. Intell. Rev.**, vol. 60, no. 1, pp. 98–121, Feb. 2024.
- [14] S. Gupta, T. Zhou, and X. Chen, “Retrieval-augmented generation for AI in clinical decision making,” **J. Med. AI Informatics**, vol. 37, no. 2, pp. 33–49, Jan. 2024.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)**, vol. 30, pp. 5998–6008, Dec. 2017.
- [16] M. Jeong, J. Sohn, M. Sung, J. Kang, “Improving Medical Reasoning through Retrieval and Self-Reflection with Retrieval-Augmented Large Language Models,” *arXiv preprint arXiv:2401.15269*, Jan. 2024.
- [17] X. Zhao, S. Liu, S.-Y. Yang, C. Miao, “MedRAG: Enhancing Retrieval-augmented Generation with Knowledge Graph-Elicited Reasoning for Healthcare Copilot,” *arXiv preprint arXiv:2502.04413*, Feb. 2025.
- [18] M. Abo El-Enen, “A survey on retrieval-augmentation generation (RAG): Domain-specific nuances and emerging frameworks,” **Neural Computing & Applications**, 2025.
- [19] O. K. Gargari, “Enhancing medical AI with retrieval-augmented generation,” *PMCID PMC12059965*, 2025.
- [20] J. Ke, “Retrieval-augmented generation for 10 large language models in preoperative instructions: accuracy, consistency, safety,” *npj Digital Medicine*, 2025.